KAYKAY'S

ISMS
System Documentation

Integrated Shop Management System: staff app, customer website, dial-code ordering and the Supabase backend

Version	2.0.0 (matches app release v2.0.0)
Date	6 November 2026
Repository	github.com/KIMGITO/ISMS
Platforms	Android · Windows · Linux · Web

Contents

1. Introduction
2. System overview
3. Technology stack
4. Repository structure
5. Roles and access control
6. Staff app: user guide
7. Customer website
8. Dial-code (USSD) ordering
9. Payments and M-Pesa
10. Customers, loyalty, wallets and debts
11. Inventory and production
12. Notifications and messaging
13. AI features
14. Receipts and printing
15. Frontend architecture
16. Database design
17. Edge functions reference
18. Security model
19. Local development setup
20. Building and releasing
21. Automatic updates
22. Deployment and operations
23. Testing
24. Known limitations and current status
25. Troubleshooting

Appendices: A. Environment variables and secrets · B. Permission codes · C. Database migrations · D. Glossary

1. Introduction

1.1 Purpose

This document describes what ISMS does, how it is built, and how to run, deploy and maintain it. It brings together material that was previously spread across the README, the `docs/` folder, the website README and the notifications notes, and checks it against the code as of release v2.0.0.

1.2 Scope

ISMS covers the whole of KayKay's day-to-day operation:

- **Selling** at the counter (full POS and a keyboard-driven Quick Sale screen), by delivery, online through the website, and by dial code from phones without internet.
- **Getting paid** in cash, by card, bank, M-Pesa (prompts and direct till payments), from a customer's wallet, on credit, or with loyalty points.
- **Stock**: products, restocks, wastage, stock counts, expiry and low-stock alerts.
- **Production**: recipes and bills of materials that turn raw milk into finished goods.
- **Customers**: profiles, loyalty tiers and points, wallets, debts, delivery schedules, duplicates and merging, and contact preferences.
- **Staff**: invitations, roles and permissions, shifts and clock-in, schedules, and rider deliveries.
- **Money**: expenses, payments, money assets per payment method, and an audit trail.
- **Communication**: notifications and push alerts for staff; SMS, WhatsApp and email to customers.
- **AI assistance**: a workspace assistant for staff, business analysis, complaint replies, and a restricted shop assistant for customers.
- **Operations**: releases, automatic updates, backups, rate limits and crash reports.

1.3 Audience

Reader	Most useful sections
Business owner or manager	2, 5, 6, 9, 10, 24
Staff trainer	5, 6, 11, 14
Developer joining the project	2–4, 15–19, 23
Whoever deploys and operates the system	18–22, 25, Appendix A

1.4 Conventions

- Screen names and buttons are in **bold**, for example **Business Management > M-Pesa**.
- File paths, table names, functions and commands are in `code`.
- Money is in Kenyan shillings (Ksh) unless stated. Times for scheduled jobs are Nairobi time (UTC+3).

- "Owner" includes Admin unless a section says otherwise.

2. System overview

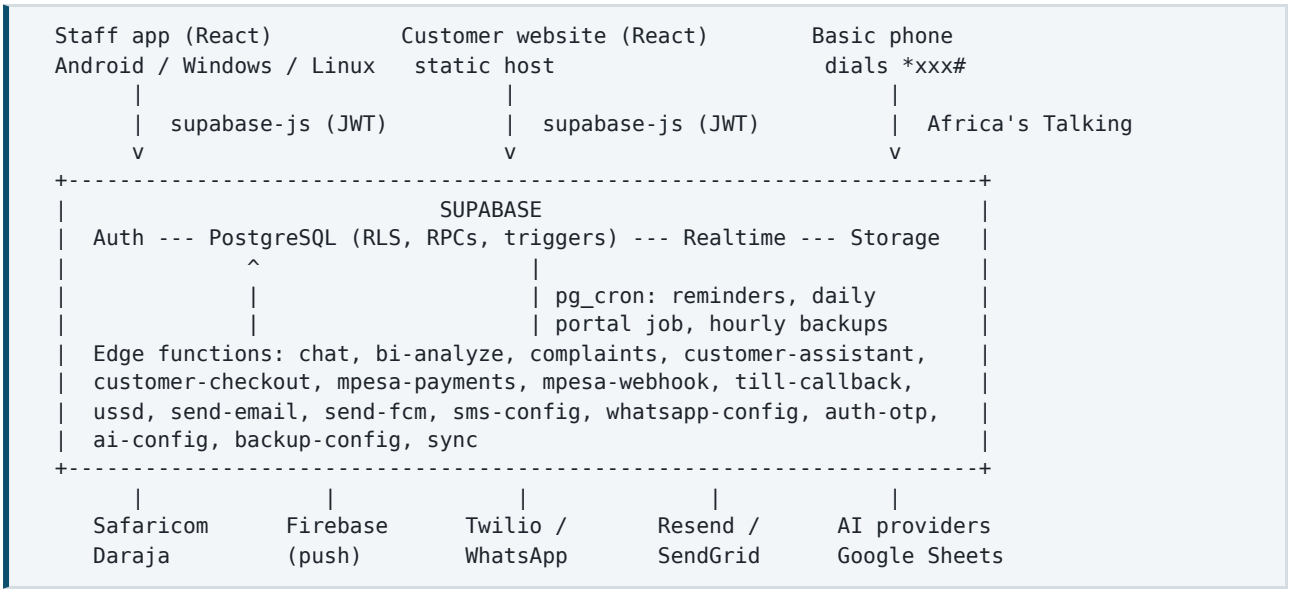
2.1 The parts

ISMS is one Supabase project with four ways in:

Part	Who uses it	Runs on	Built from
Staff app	Owners, managers, cashiers, riders	Android phones and tablets, Windows and Linux PCs	`src/`, wrapped by Capacitor (`android/`) and Tauri (`src-tauri/`)
Customer website	Customers	Any static web host (planned domain: kaykays.com)	`website/`
Dial-code ordering	Customers on basic phones	Africa's Talking USSD gateway calling an edge function	`supabase/functions/ussd/`
Backend	All of the above	Supabase: PostgreSQL, Auth, Storage, Realtime, Edge Functions, `pg_cron`	`supabase/`

No server of our own is needed. Safaricom, Africa's Talking and the Supabase notification webhook call edge functions directly.

2.2 How the parts connect



2.3 Design principles

1. **The database is the authority.** Prices, stock, points, balances and permissions are calculated and checked in PostgreSQL. The app hides what a role may not use, but the database refuses it regardless.

2. **Money moves through functions, not table writes.** Sales, wallet top-ups, debt payments, point redemptions and M-Pesa results go through `SECURITY DEFINER` functions that check the caller and update every affected balance in one transaction.
3. **Nothing financial is deleted.** Payments and expenses are voided with a reason; hard deletes on products are trapped.
4. **Real time across devices.** Changes reach every open device through Supabase Realtime, so two tills never disagree about stock.
5. **Keep selling when the network drops.** Sales made offline are queued and sent when the connection returns.
6. **Secrets stay on the server.** API keys for M-Pesa, SMS, WhatsApp, email and AI live in Supabase secrets or settings tables that only edge functions read. The app only ever sees masked values.

3. Technology stack

Layer	Technology	Notes
UI	React 19, TypeScript 5.8	Single-page app
Build	Vite 6	Staff app on port 3001; website on 5180
Styling	Tailwind CSS 4	Staff styles in `src/index.css`; website look in `website/src/theme.css`
State	Zustand 5	One store per domain, combined in `appStore.ts`
Charts	Recharts 3	Dashboard and reports
Animation	Motion 12	
Icons	lucide-react	
PDF	jsPDF 4	Receipts and exports
QR codes	qrcode	Receipts and verification
Backend	Supabase (PostgreSQL 15)	RLS on every table
Server code	Deno edge functions	One folder per function, shared helpers in `functions/shared/`
Scheduling	`pg_cron`, `pg_net`	Registered by migration `20261105012`
Desktop	Tauri 2 (Rust)	Windows `.exe` / `.msi`, Linux AppImage/ `.deb`; updater plugin
Mobile	Capacitor 8	Android; custom Java plugins for Bluetooth printing and updates
Push	Firebase Cloud Messaging	Through `send-fcm`
Payments	Safaricom Daraja	STK push and C2B (till)
USSD	Africa's Talking	
CI/CD	GitHub Actions	`build.yml`, `release-and-deploy.yml`

Version pinning rule. The Tauri npm packages (`@tauri-apps/api`, plugins) must stay on the same minor version as the Rust crates in `src-tauri/Cargo.toml`, or `tauri build` fails. This is why `package.json` pins them with `~`.

4. Repository structure

```
ISMS/
├── src/                        Staff app (React)
│   ├── App.tsx                 Shell, navigation, onboarding, tab routing
│   ├── main.tsx                Entry point
│   ├── components/             Shared widgets (ClockBar, GlobalSearch, UpdateBar,
│   │                           MpesaPaymentsPanel, DuplicateCustomers, EndShiftModal...)
│   ├── features/               One file per screen (POSView, InventoryView, ...)
│   ├── core/native/            Platform services: Bluetooth, camera, files, printer,
│   │                           notifications, secure storage, updates
│   ├── hooks/                  useForm, useOverlay, useDevicePermission...
│   ├── services/               Supabase client, repositories, notifications,
│   │                           printer/, receipt/, upload/, crash reporter
│   ├── stores/                 Zustand stores (auth, business, cart, customer,
│   │                           inventory, transactions, notifications, ...)
│   ├── utils/                  permissions.ts, phoneUtils, semver, userError...
│   └── test/runTests.ts        Test suite (npm test)
├── website/                    Customer website (separate Vite build)
│   ├── src/pages/              Home, Shop, Product, Cart, Account, Articles...
│   ├── src/seo.ts              Titles, descriptions, structured data
│   └── scripts/prerender.ts    Writes one HTML file per public page
├── supabase/
│   ├── migrations/             32 SQL migrations (see Appendix C)
│   ├── functions/              17 edge functions + shared/
│   ├── recalculate_loyalty_points.sql
│   ├── dev_clear.sql           Wipes data, keeps schema (development only)
│   └── dev_reset.sql           Drops schema and data (development only)
├── android/                    Capacitor Android project
│   └── app/src/main/java/com/kaykays/app/
│       ├── AppUpdatePlugin.java        Download, verify and install updates
│       └── BluetoothSerialPlugin.java  Classic Bluetooth printers
├── src-tauri/                  Tauri desktop shell (Rust)
├── scripts/                    release-version, update-manifest, stamp-version,
│                               sync-notification-sounds
├── .github/workflows/          build.yml, release-and-deploy.yml
├── docs/                       Guides, including this document
├── public/                     Icons, logo, notification sounds
├── capacitor.config.ts
├── vite.config.ts
└── package.json
```

4.1 npm scripts

Command	What it does
`npm run dev`	Staff app in the browser at `http://localhost:3001`
`npm run tauri:dev`	Staff app in a desktop window with hot reload
`npm run android`	Build, sync to Android and assemble a debug APK
`npm run android:cap`	Sync sounds, build, and `cap sync android`
`npm run build`	Production web bundle in `dist/`
`npm run tauri:build`	Desktop installers for the current OS
`npm run lint`	TypeScript check of the staff app
`npm test`	Run `src/test/runTests.ts`
`npm run website:dev`	Website at `http://localhost:5180`

Command	What it does
`npm run website:build`	Website build and prerender into `website/dist`
`npm run website:lint`	TypeScript check of the website
`npm run sounds:sync`	Copy notification sounds into the Android project

5. Roles and access control

5.1 Three layers

A role's access is enforced in three places, so hiding a screen is never the only protection:

Layer	What it does	Where
App	Hides screens and buttons	`SYSTEM_ROLES` in `src/utils/permissions.ts`
Database	Refuses rows and functions, even for direct API calls	`has_permission()`, migration `20261105014`
Edge functions	Refuse the call before doing paid or sensitive work	`requirePermission()` in `supabase/functions/shared/cors.ts`

5.2 Roles

Role	Purpose
Owner / Admin	Full control, including settings, API keys, staff and deletion. Bypass permission checks by design.
Manager	Runs the business day to day: sales, stock, customers, expenses, staff schedules, reports. No API keys, no deletes of key records.
Cashier	Sells, opens and closes shifts, registers customers, takes payments and debt repayments.
Rider	Sees and completes only the deliveries assigned to them.

Other database roles, such as `Sales Staff` and `Viewer`, exist but have no permissions until an Owner grants some.

5.3 Who can see what

Area	Owner, Admin	Manager	Cashier	Rider
Own profile, own shifts, clock in	Yes	Yes	Yes	Yes
Home (business figures)	Yes	Yes	No	No
Sell	Yes	Yes	Yes	No
Inventory	Full	Full	View	No
Sales history	All	All	Own sales and customers' sales	No
Online orders	All	All	View, prepare	Only assigned deliveries
Customers	Full	Full, no delete	View, add	Only customers on their deliveries

Area	Owner, Admin	Manager	Cashier	Rider
Debts	Yes	Yes	Yes	No
Profit	Yes	Yes	No	No
Complaints	Reply	Reply	View, log	No
Expenses	Yes	Yes, no delete	No	No
Staff	Full	View, schedule	No	No
Settings, backups, API keys	Yes	Some settings, no keys	No	No
Set points or tiers by hand	Yes	Yes	No	No
Set wallet or debt balances by hand	Yes	Yes	No	No
Deliveries assigned to them	Yes	Yes	Yes, by assigning	Yes

5.4 Changing access

An Owner or Admin can grant or remove any permission per role in the **Roles** screen. Changes save to `role_permissions`, which both the app and the database read, and override the defaults.

To change the defaults themselves, edit `SYSTEM_ROLES` in `src/utils/permissions.ts` and `role_default_permissions` together, in a new migration. `npm test` fails if the two differ (test "Database role permissions match the app").

The full list of permission codes is in Appendix B.

5.5 Riders

A rider receives deliveries only through `get_my_deliveries`, which returns just what the job needs (name, phone, location, items and amount) for their own deliveries. Staff assign online orders in **Online Store**; a till sale links to a rider when the rider is chosen by name. A delivery with a typed-in rider name isn't linked to an account, so nobody sees it under **Deliveries**.

6. Staff app: user guide

6.1 Getting started

First run. The first person to sign up creates a business (name, type, country, currency) and becomes its Owner through `create_business_with_owner`. Supported countries are Kenya, Uganda, Tanzania, Rwanda and Somalia; currencies Ksh, USh, TSh, RWF and USD. Business types include retail, wholesale, bakery, restaurant, dairy farm, milk processing plant, agrovet, pharmacy and others.

Inviting staff. In **Workers**, the Owner invites a person by email with a role. The invitee receives a code, signs up or signs in, and joins through `accept_invitation_with_token`. Invitations can be revoked and expire.

Signing in and recovering a password. Email verification and password reset codes are issued and checked by the `auth-otp` edge function. Codes are six digits, never returned to the app, and limited to five wrong guesses each.

Switching business. A person can belong to several businesses and switch between them; all data is scoped to the business selected.

6.2 Navigation

On desktop, screens are in a sidebar. On phones, a bottom bar holds at most five slots, with **Sell** in the centre for cashiers and **Settings** moved under **More**. Each screen requires a permission and disappears for roles without it.

Screen	Label	Required permission	Purpose
Home	Home	`home.view`	Business figures at a glance
Shift dashboard	Shift	`dashboard.view`	Current shift, staff, today's activity
Quick Sale	Sell	`pos.create_sale`	Fast keyboard and scanner checkout
Point of sale	POS	`pos.create_sale`	Full checkout with customers and options
Inventory	Inventory	`inventory.view`	Products and stock
Sales log	Sales Log	`orders.view`	Past sales and receipts
Customers	Loyalty	`customers.view`	Profiles, wallets, debts, points
Feedback	Feedback	`complaints.view`	Reviews, complaints, messaging
Online Store	Online Store	`online_store.view`	Website orders, messages, guides, settings
Deliveries	Deliveries	`deliveries.view`	A rider's assigned deliveries
Workers	Workers	`staff.view`	Staff, invitations, schedules
Roles	Roles	`staff.roles`	Permission matrix
Assistant	Assistant	`ai.use`	Workspace AI assistant
Profile	Profile	`dashboard.view`	Own account, shifts and settings

Screen	Label	Required permission	Purpose
Business	Business	`business.update`	Details, expenses, payments, M-Pesa
Production	BOM	`bom.view`	Recipes and production batches
Settings	Settings	`settings.view`	Appearance, receipts, printers, integrations
About	About	—	Version and **Check for updates**

A global search box finds products, customers and sales from anywhere. The **Pending actions** drawer holds actions proposed by the AI assistant for a person to check and approve.

6.3 Shifts and clock-in

The clock bar at the top lets staff clock in and out. Opening a shift records the starting cash; every sale made by that employee adds to the shift's count and total (done in the database trigger). **End shift** asks for the closing cash, shows the difference, and produces a shift report that can be sent as a notification to the Owner.

Staff presence is shown as online (seen in the last 3 minutes), away (3–10 minutes) or offline, based on `employees.last_seen_at`, which the app updates every 2 minutes.

6.4 Quick Sale

Quick Sale is built for speed at a busy counter with a keyboard or barcode scanner.

Key	Action
Type or scan, then Enter	Add the product (an exact SKU match wins)
Up / Down	Move through search results
+ / −	Change the last item's quantity
F1 / F2 / F3	Finish the sale by Cash / M-Pesa / Card
Esc	Clear the search, then the sale

6.5 Point of sale

The POS screen supports everything Quick Sale does plus:

- **Customer selection**, or a walk-in sale. Only sales to a registered customer earn points.
- **Quantity, price override and discounts**, each controlled by its own permission (`pos.override_price`, `pos.discount`).
- **Payment methods**: Cash, M-Pesa, Card, Bank, Mobile Wallet (customer wallet), Credit (adds to the customer's debt), and loyalty points.
- **Split payments**, for example part wallet and part M-Pesa.
- **Delivery**: a delivery fee, delivery zone and rider. Delivery fees never earn points.
- **Send M-Pesa prompt** to the customer's phone (see section 9).
- **Receipt**: printed, shared as PDF or image, or sent by message.

When the sale completes, `process_customer_transaction` records it. Triggers then reduce stock, add points, update the shift, raise the customer's tier if earned, and post the money to the right money asset.

6.6 Inventory

Inventory holds the product catalogue: name, category, SKU or barcode, unit, cost, sale price, tax, image, minimum stock and expiry. From here staff with the right permissions can:

- **Receive stock** (restock), **adjust stock** with a reason, run **stock counts**, and record **wastage** such as spillage or souring.
- See **stock history**: every adjustment, sale and production movement.
- **Bulk edit** categories (`update_product_category_bulk`) and stock (`adjust_stock_bulk`).
- **Import and export** products.

When stock falls to the minimum, the `fn_check_low_stock` trigger raises a **Stock Almost Finished** notification; at zero it raises **Out Of Stock**.

Products are never deleted outright: `fn_trap_product_hard_deletes` turns a delete into an archive so past sales keep their product.

6.7 Sales log

Lists past sales with their items, payment, customer, staff member and receipt number. Sales can be filtered, reprinted, exported to PDF, and refunded or cancelled with the right permission (`pos.refund`, `pos.cancel_sale`). Cashiers see only their own sales and their customers' sales.

6.8 Customers (Loyalty)

Each customer has tabs for **Overview & Profile**, **History Events**, **Wallet & Debt Ledger** and **Outreach & Messages**. From here staff can:

- Register customers and set customer type, payment terms, group, delivery schedule and notes.
- Top up the wallet (`process_wallet_topup`), take a debt payment (`process_debt_payment`, oldest debts first through `settle_customer_debt_fifo`), or send an M-Pesa prompt.
- Redeem points, or adjust points, tier, wallet or debt by hand (Owner and Manager only; every change is logged).
- Review **Duplicates** and merge two records (section 10.5).
- See each customer's account number, from 100001.

6.9 Feedback

Two tabs: **Reviews & Sentiment**, which lists complaints and ratings with filters by rating, sentiment and status, and **Communication Center**. AI can analyse a complaint's sentiment, draft a reply and improve a draft (`complaints` edge function). Replies are saved in `complaint_replies`.

6.10 Communication Center

Sends a message to a list of customers. Staff first choose the kind of message, **Order update or reminder** or **Offer or promotion**, then how to send it:

Option	What happens
Automatic	WhatsApp API, then SMS. Anyone still unreached is listed to send by hand.
WhatsApp API, SMS or Email	That channel only. Anyone it can't reach is listed to send by hand.
By hand	Lists everyone; each tap opens that customer's WhatsApp chat.

Customers who haven't agreed to offers are skipped for promotions. The SMS, WhatsApp and email functions check consent again on the server. A **Staff Reminders** feed shows the signed-in person's schedule reminders.

6.11 Online Store

Tabs: **Orders**, **Messages**, **Feedback**, **Guides** and **Website**.

- **Orders**: website and dial-code orders, their payment state, and anything flagged for attention (wrong amount, second payment, stock shortage). Staff prepare orders, assign riders (`staff_assign_rider`) and update status (`staff_update_online_order`).
- **Messages**: conversations with website customers.
- **Feedback**: reviews submitted on the website.
- **Guides**: milk guides and articles shown on the website; starter guides can be added, edited and published.
- **Website**: the site's name (`kaykays`; the site stays hidden until set), delivery fee and delivery areas, which products are shown, and loyalty settings (point value and minimum).

6.12 Deliveries

A rider sees their assigned deliveries with the customer's name, phone, location, items and amount, and marks each one out for delivery or delivered (`rider_update_delivery`). Completion notifies the Owner.

6.13 Workers and Roles

Workers lists staff with their role, branch, presence and shifts, handles invitations, and sets **schedules** (with repeat: none, daily, weekly, monthly). Staff receive a **Schedule Reminder** notification before each scheduled shift, raised by a job every 15 minutes.

Roles is the permission matrix described in section 5.4.

6.14 Business Management

Tabs: **Details**, **Expenses**, **Payments** and **M-Pesa**.

- **Details**: name, description, address, phone, email, logo and cover image. These appear on receipts and the website.
- **Expenses**: record costs by category (categories are seeded per business), with receipt photos. Expenses are voided with a reason, not deleted.
- **Payments**: every payment received, filterable by method, with void (`void_business_payment`) for mistakes. Money assets show the running balance per method (cash, M-Pesa, card, bank and so on) with a ledger of every movement.
- **M-Pesa**: the till number, **Register till with Safaricom**, and the **Paid to the till** queue (section 9).

6.15 Settings

Settings is grouped into sections:

- **Design and Branding**: theme and brand colours.
- **Messages and Codes**: message templates and number formats.
- **Printers**: add and test receipt printers (section 14).
- **Notifications**: per-category choices, sounds, and pause.
- **AI providers**: keys and models for Google Gemini, OpenAI, Anthropic Claude, DeepSeek, Groq, OpenRouter and Hugging Face. Keys are stored server-side and shown masked.
- **Twilio SMS** and **WhatsApp API** credentials.
- **Backups**: Google Sheets backup and its schedule.
- **Device permissions**: camera, location, Bluetooth, notifications and storage.
- **Local terminal security**: a personal PIN that locks the app on a shared device.

6.16 Production (BOM)

See section 11.3.

7. Customer website

7.1 What customers can do

The website (`website/`) is a separate Vite build that shares only the Supabase project. Customers can:

- Browse the shop and product pages, read milk guides and articles.
- Create an account by email, or with Google, Apple or a phone number once those are set up.
- Add to cart and pay by M-Pesa prompt, choosing a delivery area.
- See their orders, points (Rewards), schedules, messages and contact preferences under **Account**.
- Set up repeat orders (schedules), send feedback, and contact the shop.
- Ask the shop assistant about products, availability and totals.

Routes: `/`, `/shop`, `/products/...`, `/cart`, `/articles/...`, `/feedback`, `/contact`, `/account`.

7.2 How the website reaches data

Customers sign in through Supabase Auth and have no business membership, so staff policies exclude them. They read only their own rows through RLS and write only through database functions such as `place_online_order`, `save_order_schedule`, `update_my_account` and `redeem_my_points`. The shop's public data comes from `get_public_storefront`, which returns only what customers should see.

Never add anon or customer policies on `products` or `businesses`. A policy can't hide columns, so it would expose `products.cost` and other private fields.

7.3 Orders and payment

1. `place_online_order` prices everything on the server; the website's own totals are never trusted. Totals are rounded down to whole shillings because M-Pesa rejects cents.
2. `customer-checkout` (action `pay`) starts an STK push for the customer's own order. It is limited to 6 attempts per 10 minutes per customer.
3. Safaricom calls `mpesa-webhook`, which records the result. Only this callback can mark an order paid.
4. Payment inserts an ordinary `transactions` row, so the usual triggers update stock, loyalty points and money assets.
5. Wrong amounts, second payments and stock shortages are flagged for staff under **Online Store > Orders**.

Refunds are manual: staff handle them in M-Pesa and the POS. Points used on a cancelled order are returned (`fn_refund_order_points`).

7.4 Schedules

A daily job, `run_customer_portal_daily` at 05:00 Nairobi time, turns due schedules into unpaid orders and messages the customer, because M-Pesa needs their PIN for each payment. It also cancels orders left unpaid for 48 hours.

7.5 Shop assistant

`customer-assistant/logic.ts` answers availability, quantity and total questions in code, from live stock. The language model only sees general guides and the customer's text, never stock or cost figures, and its replies are checked before the customer sees them. Usage is capped per customer (`assistant_usage`, `consume_assistant_quota`).

7.6 Search engines and sharing

`website/src/seo.ts` holds each page's title, description and structured data (Store, Product, Article, BreadcrumbList), generated from real business data. `scripts/prerender.ts` writes a separate HTML file per public page at build time, so search engines and link previews see the right content. Cart and account pages are never indexed. Until `VITE_SITE_URL` is set, every page asks search engines not to index it.

Rebuild and upload the site after adding products or guides or changing prices so search results stay current. Customers always see live data either way.

7.7 Look and photos

`website/src/theme.css` holds the whole public look: colours, fonts (Italiana and Manrope), buttons and fields. The home page hero is a carousel (`HeroCarousel.tsx`). Visitors whose device asks for reduced motion get a still page. Temporary Unsplash photos in `src/images.ts` carry credits in the footer and are replaced as soon as a product, guide or business has its own image.

7.8 Going live

1. Apply the migrations, including `20261105002_security_hardening.sql` and `20261105010_customer_portal.sql`.
2. Set the M-Pesa secrets, starting with `MPESA_ENV=sandbox` (Appendix A).
3. Deploy `customer-checkout`, `mpesa-webhook` and `customer-assistant`.
4. In Supabase Auth, add the website address to the redirect URLs and set up an email provider.
5. In **Online Store > Website**, set the website name to `kaykays`, set the delivery fee or delivery areas, and untick products customers shouldn't see.
6. Add, check and publish the starter guides under **Online Store > Guides**.
7. Fill in the business description, address, phone and email in **Business Management**.
8. Run `npm run website:build` and upload `website/dist`. The host must serve `index.html` for unknown paths; `_redirects` (Netlify, Cloudflare Pages) and `404.html` (GitHub Pages) are included.

9. Make a sandbox payment, then switch to production till credentials and make one small real payment before announcing the site.

To connect kaykays.com: point the domain at the host with HTTPS, set `VITE_SITE_URL=https://kaykays.com`, rebuild and upload, add `https://kaykays.com/account` to the Supabase redirect URLs, and submit `https://kaykays.com/sitemap.xml` in Google Search Console.

7.9 Sign-in options

Email works without further setup. The Google button shows by default and explains that it isn't available until set up. For phone and Apple, finish the provider setup, add the option to `VITE_AUTH_PROVIDERS` (for example `google,phone,apple`), and rebuild.

Google and Apple both return customers to `https://<project-ref>.supabase.co/auth/v1/callback`. Customers who sign in with Google, Apple or phone are asked for any missing name and M-Pesa number the first time. Apple's client secret must be renewed every six months, or Apple sign-in stops. The detailed provider steps are in `docs/INTEGRATIONS.md`.

8. Dial-code (USSD) ordering

8.1 Purpose

Dial-code ordering lets customers without a smartphone or internet order and pay from any phone, through Africa's Talking.

8.2 What a caller can do

- Create an account with just their name.
- Order from the first seven products in stock.
- Pay by M-Pesa prompt (the line must have M-Pesa).
- Check their orders, points and balances.

The mobile network supplies the phone number, so it identifies the caller. Someone with a website account under another login gets a separate dial-code account.

8.3 How it works

Africa's Talking sends `sessionId`, `serviceCode`, `phoneNumber` and `text` on every step, with each answer so far joined by `*`. The menu is a pure function in `ussd/menu.ts`, so it can be tested without a gateway. Each reply starts with `CON` to continue or `END` to finish. Requests are authorised by `USSD_CALLBACK_TOKEN` in the callback URL and limited to 40 per 10 minutes per phone.

The gateway expects a reply within 10 seconds. Placing an order calls M-Pesa inside that time, so a slow M-Pesa response can show the caller a network error. This must be tested on the live service.

8.4 Setting it up

Sandbox (free):

1. Create an Africa's Talking account and open the sandbox (username `sandbox`).
2. Generate a token with `openssl rand -hex 32`, then:

```
bash supabase secrets set USSD_CALLBACK_TOKEN=<token> USSD_SHOP_SLUG=kaykays supabase functions deploy ussd
```

3. Under **USSD > Create Channel**, set the callback to `https://<project-ref>.supabase.co/functions/v1/ussd?token=<token>`.
4. Dial it on the Africa's Talking simulator.

Live: request a service code (a shared code such as `*384*123#` is the usual start; approval can take weeks), top up the account, set the same callback on the live channel, and test one order from a real phone.

9. Payments and M-Pesa

9.1 Payment methods

Method	Effect
Cash, Card, Bank, M-Pesa	Added to that method's money asset
Mobile Wallet	Taken from the customer's wallet balance
Credit	Added to the customer's debt; no money asset changes until repaid
Points	Taken from the customer's points at the configured value

Each money asset (`money_assets`) keeps a running balance per business and method. Every change is written to `payment_asset_ledger` with the balance before and after, so the history can always be reconstructed. Expenses reduce the asset they were paid from (`fn_process_expense_asset_update`).

9.2 M-Pesa: a till, not a paybill

KayKay's uses an M-Pesa **till** (Buy Goods). A payment a customer sends to a till from their own phone carries **no account number**, so it can't be matched to a customer automatically. The system handles M-Pesa three ways:

How the customer pays	What happens
Online order or dial code	A prompt goes to their phone; the order settles when they pay.
Staff press **Send M-Pesa prompt**	A prompt goes to their phone. When they pay, the payment is recorded automatically, against their debt first and then their wallet, or as a walk-in.
They pay the till from their own phone	The payment waits under **Paid to the till** until staff assign it to a customer, mark it as a sale already rung up, record it as a walk-in, or ignore it with a note.

9.3 The functions involved

Function	Called by	Role
`mpesa-payments`	Staff app	`status`, `save_till`, `register` (tell Safaricom where to send till payments), `request` (send a prompt), `check` (ask about a prompt with no answer)
`customer-checkout`	Website	Prompt for a customer's own order
`mpesa-webhook`	Safaricom	Result of every STK prompt, matched by CheckoutRequestID
`till-callback`	Safaricom	Direct till payments (C2B): `?type=validation` and `?type=confirmation`

Function	Called by	Role
`process_business_payment`	Database	Records a matched payment against debt, then wallet
`record_till_payment`, `handle_till_payment`	Database	Queue and resolve direct till payments
`log_mpesa_payment`	Database	Logs the payment and raises **Payment Received**

Safaricom refuses callback URLs containing words like "mpesa", which is why the C2B function is called `till-callback`.

9.4 Account numbers

Every customer has an account number from 100001 (a global sequence, kept through merges by `fn_keep_customer_account_number`). It appears on the customer's page and goes out as the account reference on prompts. If the business ever adds a paybill, payments quoting the number can be matched automatically.

9.5 Rate limits on prompts

Staff prompts are limited to 60 per hour per person and 600 per day per business; website checkout to 6 per 10 minutes per customer.

9.6 Setting up M-Pesa

1. Create an app at developer.safaricom.co.ke and make a sandbox payment first.
2. Apply to go live for the till. Safaricom issues the store number and passkey.
3. Set the secrets:

```
bash supabase secrets set MPESA_ENV=production MPESA_CONSUMER_KEY=... \
MPESA_CONSUMER_SECRET=... MPESA_SHORTCODE=<store number> \ MPESA_PARTY_B=<till number>
MPESA_PASSKEY=... \ MPESA_TRANSACTION_TYPE=CustomerBuyGoodsOnline \ MPESA_CALLBACK_TOKEN=
<long random string>
```

4. Deploy `mpesa-webhook`, `mpesa-payments` and `till-callback`, and apply migration `20261106001`.
5. In **Business Management > M-Pesa**, enter the till number and press **Register till with Safaricom**. Each branch with its own till does this once.

Set `MPESA_C2B_SHORTCODE` if Safaricom registers the till under a different short code from `MPESA_SHORTCODE`. In the sandbox, Daraja's test short code is a paybill, so use `CustomerPayBillOnline` and leave `MPESA_PARTY_B` empty.

9.7 Voiding

Payments are voided, never deleted (`DELETE` is revoked on the payment tables). A void records who, when and why, and reverses the money asset.

10. Customers, loyalty, wallets and debts

10.1 Points

- Registered customers earn **1 point per Ksh 100** spent, excluding delivery. Walk-in sales earn nothing.
- **One point is worth Ksh 1**, redeemable from 100 points. This is about 1%, in line with common Kenyan supermarket schemes. The value and minimum are set in **Online Store > Settings** (`loyalty_settings`).
- Points can come off a till sale (`spend_points_at_till`) or website order, or be moved onto the account. They can't be exchanged for cash.
- Points used on a cancelled order come back. A website order must leave at least Ksh 1 to pay by M-Pesa.
- To rebuild balances from sales history, run `supabase/recalculate_loyalty_points.sql`. Older app versions still award extra points, so run it again once everyone has updated.

10.2 Tiers

Tier	Total spend	Equivalent points
Bronze	Ksh 0 – 9,999	0 – 99
Silver	Ksh 10,000 – 29,999	100 – 299
Gold	Ksh 30,000 and above	300 and above

`calculate_customer_tier` sets the tier; `fn_raise_customer_tier` only ever raises it automatically. Owners and Managers can set it by hand.

10.3 Wallets

A customer can deposit money in advance (`process_wallet_topup`) and spend it at the till (`process_wallet_spend`). Every movement is in `wallet_transactions` and the customer ledger.

10.4 Debts

A sale paid by Credit adds to the customer's debt. Repayments (`process_debt_payment`) clear the oldest debts first (`settle_customer_debt_fifo`) and are recorded in `debt_payments` and `credit_payments`. `v_open_debts` lists customers with balances, and a **Debt Due Reminder** notification can be raised.

Balances can't be changed by direct table writes: `fn_guard_customer_balances` refuses them unless they come through the proper function or from an Owner or Manager adjusting by hand.

10.5 Duplicate customers and merging

A customer added by staff and the same person signing up online start as two records. They are never linked automatically, because a phone number typed at sign-up isn't proof of who someone is. Staff with `customers.manage` review matches under **Customers > Duplicates** (`find_duplicate_customers`), and are told when a sign-up uses a number already on file (`fn_flag_duplicate_signup`). Pairs that aren't duplicates can be dismissed.

Data	When merged (`merge_customers`)
Debt, wallet, points, purchases	Added together, then wallet money clears debt
Sales, payments, ledger, online logins	Move to the kept record
Tier	The higher one, or higher still if the combined points earn it
Customer type, payment terms, delivery schedule, notes	The kept record's
Name, phone, email	Staff choose
Order updates	Off if either record turned them off
Offers	On if either record agreed

Each merge is saved in `customer_merges` with both records as they were. There is no undo button. A device still using the old record has its new sales moved to the kept record (`fn_redirect_merged_customer`), and its balance changes refused until it refreshes (`fn_block_merged_customer_balances`).

Planned: once SMS codes can verify phone numbers, automatic linking will be added only for one matching individual with no debt or wallet balance. Institutions, numbers on several records, and records with a balance will stay with staff, because numbers get reassigned.

10.6 Contact preferences

Customers choose in their website profile, and staff can record the choice on the customer's form. Order updates and reminders are on by default; offers need the customer's agreement. `fn_contact_allowed` is checked by every sending function.

10.7 Customer groups

Migration `20261105013` groups customers by customer type, payment terms and delivery rhythm, set on the customer form. The same migration moved tier changes to the server, added point redemption, and allowed sign-ups without an email (phone or dial code). Staff can also pick items to follow in reports (`report_tracked_items`).

11. Inventory and production

11.1 Stock movements

Movement	Recorded in	Effect
Sale	`transaction_items`	Stock down (`fn_process_transaction_item`)
Restock	`inventory_adjustments` (Restock)	Stock up
Damage or wastage	`inventory_adjustments` (Damage)	Stock down
Stock count	`inventory_adjustments` (Reconciliation)	Stock set to the counted figure
Purchase from supplier	`purchases`, `purchase_items`	Stock up when received
Production batch	`production_batches`	Raw materials down, finished goods up

`adjust_stock_manually` and `adjust_stock_bulk` check permissions and log who changed what.

11.2 Suppliers and purchases

Suppliers (for example raw milk cooperatives) are kept in `suppliers`. Purchases record what was ordered and received, with approval and cancellation controlled by the `purchases.*` permissions.

11.3 Bill of materials and production batches

Recipes turn raw materials (milk) into finished goods (yogurt, mala and so on).

- A **bill of materials** (`bill_of_materials`, `bom_ingredients`) links a finished product to the ingredients and quantities per unit.
- **Creating a batch** (`fn_create_production_batch`) works out what is needed, refuses if stock is short, and deducts the raw materials.
- **Completing a batch** (`fn_complete_production_batch`) adds the finished goods.
- **Cancelling a batch** (`fn_cancel_production_batch`) puts unused raw materials back and records the rest as waste.

All three run atomically in the database, so a failed step changes nothing. The calculations are covered by tests ("BOM Batch Ingredient Requirement & Stock Shortage Calculation", "Batch Cancellation Restocking vs Waste Split Logic").

11.4 Assets

`business_assets` records the business's equipment (cooler, motorbike, separator), viewable and managed under the `assets.*` permissions.

12. Notifications and messaging

12.1 How a notification travels

1. Something happens: a trigger (low stock), a database function (M-Pesa payment), a scheduled job (reminders), or the app (shift closed).
2. A row is inserted into `notifications` with its type, priority, payload and target user or role.
3. A Supabase database webhook on `INSERT` calls `send-fcm`.
4. `send-fcm` asks `fn_notification_recipients` who should get it, honouring each person's preferences, then pushes through Firebase to their registered devices (`device_fcm_tokens`). Dead tokens are removed automatically.
5. The row's status moves from `pending` to `sent` (or `delivered` when no device token exists).
6. Open apps receive the row through Realtime and show it in the inbox with a sound and a banner.

`send-fcm` only accepts calls carrying the service role key or `FCM_WEBHOOK_SECRET`. **Before deploying it, confirm the webhook sends one of those, or pushes stop.**

12.2 Who gets what

`fn_notification_catalog()` is the single list of categories. Each person's choices are stored in `notification_preferences`, so they follow them across devices.

Category	Who can have it
Stock	`inventory.view`
Sales reports	`reports.sales`
Debts	`customers.debts`
Deliveries	`deliveries.view`, and anyone a delivery is assigned to
Payments	`payments.view` or `payments.verify`
Online store	`online_store.view`
AI insights	`ai.insights`
Schedules, announcements	Everyone, limited to the role named on the notification
Account and security	Everyone; can't be turned off

Everything is on by default for those who can have it. Pausing turns off everything except account and security alerts.

12.3 Notification types

Type	Source	Priority
Schedule Reminder	`flag_due_schedule_reminders()` every 15 minutes	high
Out Of Stock	`fn_check_low_stock` trigger	critical
Stock Almost Finished	`fn_check_low_stock` trigger	high
Payment Received	`log_mpesa_payment`	high
Custom Notification	App: shift open/close, clock in/out, end-of-shift report	caller sets
AI Business Insight, AI Recommendation, AI Risk Alert	AI analysis	medium / high
Delivery Assigned, Delivery Completed	Delivery flow	medium
Sales Summary, Daily, Weekly, Monthly Report	Report generators	medium
Debt Due Reminder	App	high
Low Cash Balance	Cash drawer monitor	medium
Role Invitation	Invitation flow	high
Account Activity	Auth and security	medium
System Update	Release pipeline	low
Business Announcement	Owner broadcast	low

12.4 The notification screens

Notifications has a category panel (All Activity, Stock & Inventory, Sales & Reports, Deliveries, Payments, AI Insights, Schedule Reminders, System & Security), tabs for All, Unread, Important and Reminders, search, and delivery status badges.

12.5 Android specifics

Android uses notification channels `default`, `alerts` and `shifts`, a monochrome milk-glass status icon (`ic_stat_name`), and amber (`#F59E0B`) as the notification colour. Local reminders are scheduled on the device with `@capacitor/local-notifications`. Twelve notification sounds are bundled and copied into the Android project by `npm run sounds:sync`.

12.6 Customer messaging channels

Channel	Function	Notes
SMS	`sms-config` (Twilio)	200 per hour per person, 1,000 per day per business
WhatsApp	`whatsapp-config` (Meta Cloud API)	Free text only within 24 hours of the customer's last message; otherwise an approved template whose body is one variable. Delivery reports aren't tracked.

Channel	Function	Notes
Email	`send-email` (Resend or SendGrid)	30 per hour per person; branded HTML template
Website messages	`customer_messages`	`staff_send_customer_message`, `send_customer_message`

13. AI features

13.1 Providers

AI calls go through `supabase/functions/shared/ai-runner.ts`, which supports Google Gemini, OpenAI, Anthropic Claude, DeepSeek, Groq, OpenRouter and Hugging Face (the default provider, with token rotation). The Owner chooses the provider and model and enters keys in **Settings**; the `ai-config` function stores them in `ai_settings` and only ever returns masked keys.

13.2 Features

Feature	Function	Permission	What it does
Workspace assistant ("Kim" by default, set by `VITE_AI_NAME`)	`chat`	`ai.use`	Answers questions about catalogue, stock, customers and staff in plain text; can propose actions
Business analysis	`bi-analyze`	`ai.insights` or `reports.view`	Turns dashboard figures into a structured report with insights, recommendations and risks
Complaints	`complaints`	`complaints.view`	`analyze`, `generate_reply`, `improve_reply`, `analytics`
Shop assistant	`customer-assistant`	Public, quota per customer	Restricted answers for website customers (section 7.5)

13.3 Human in the loop

The workspace assistant never changes data on its own. Actions it suggests, such as creating a schedule, adjusting stock or preparing a checkout, go to the **Pending actions** drawer (`pendingActionStore.ts`). A person with the required permission checks the details, edits them if needed, and approves or rejects. The action then runs with that person's permissions, not the AI's.

13.4 Limits

AI is limited to 20 requests per minute per person and 500 per day per business. Without a configured provider, AI answers are unavailable and the rest of the app works normally.

14. Receipts and printing

14.1 Receipts

`src/services/receipt/` builds receipts from templates: **classic**, **compact**, **modern**, **retail** and **milkShop**. Receipt settings (`receipt_settings`) control paper size, logo, header and footer text, and whether to include a QR code and barcode. A receipt can be:

- Printed on a thermal printer (ESC/POS).
- Exported to PDF (jsPDF) or an image and shared through the device's share sheet or WhatsApp.
- Verified later by scanning its QR code (`receipt_verifications`).

Receipt and customer numbers are formatted by tested helpers ("formatReceiptNumber Formatter", "formatCustomerNumber Formatter").

14.2 Printers

`src/services/printer/` contains a printer service with a print queue, a connection manager and an ESC/POS generator.

- **Android**: pair the printer once in Android Bluetooth settings; it then appears under printer settings in the app. Classic Bluetooth serial printing is implemented in `BluetoothSerialPlugin.java`.
- **Desktop and browser**: Web Bluetooth where the browser supports it, otherwise the system print dialog.

Printer settings are stored per business in `printer_settings`. The **Test print** button checks the connection.

15. Frontend architecture

15.1 Layers

```
features/*View.tsx        Screens
      |
components/               Shared widgets
      |
stores/ (Zustand)         State and actions per domain; appStore.ts combines them
      |
services/                 repositories.ts, supabaseService.ts, notifications,
      |                   printer, receipt, upload, network, realtime
core/native/              Platform services (Capacitor / Tauri / browser)
      |
Supabase client           supabaseClient.ts (auth, RPC, Realtime, Storage)
```

15.2 Stores

Store	Responsibility
`authStore`	Sign-in, sign-up, onboarding, memberships, the current employee, shifts
`businessStore`	Businesses, branches, switching, business settings
`cartStore`	The sale being built: items, quantities, discounts, payment, delivery
`transactionStore`	Completed sales and their sync state
`inventoryStore`	Products, categories, adjustments, wastage
`customerStore`	Customers, wallets, debts, points
`extraModulesStore`	Purchases, production, recipes, assets, audit logs, expenses
`notificationStore`	Inbox, unread counts, toasts, schedule reminders
`pendingActionStore`	AI-proposed actions awaiting approval
`uiStore`, `overlayStore`	Active screen, modals, chat history
`updateStore`	Update state for the update bar
`systemHealthStore`	Connection and service health
`appStore`	One facade over all of the above

15.3 Real time

`realtimeService.ts` subscribes to Supabase Realtime for the current business. When a row changes on another device, such as a sale reducing stock, the relevant store updates and the screen re-renders.

15.4 Offline sales

`networkService.ts` tracks whether the device is online. A sale made while offline is kept on the device with status `Offline_Pending` and is sent once the connection returns. The `sync` edge function and `offline_sync_queue` table keep queued sales safe across restarts and

device switches. Prices and stock are re-checked by the database when the sale arrives.

15.5 Uploads

`services/upload/` follows a ports-and-adapters design: domain entities and ports, a Supabase upload client, an offline upload queue, and a `ProcessUploadUseCase`. Images go to the Storage buckets in section 16.7.

15.6 Native layer

`src/core/native/` wraps platform features behind one interface each, so screens don't care whether they run on Android, desktop or in a browser: Bluetooth, camera, device info, files, geolocation, notifications, permissions, printer, secure storage, share, splash screen, status bar and updates.

15.7 Errors shown to people

`utils/userError.ts` (`toUserMessage`) turns technical errors into plain sentences and keeps friendly ones as they are, so staff never see stack traces or SQL. Crashes are reported separately (section 22.5).

16. Database design

16.1 Tenancy

Every business table carries `business_id`. Row-level security policies let a signed-in person reach only rows of businesses they are an active member of (`get_user_business_ids`, `assert_business_member`). Functions that act on a business check membership and permission first (`assert_permission`, `has_permission`).

16.2 Tables by area

Area	Tables
Identity and access	`businesses`, `branches`, `users`, `employees`, `business_memberships`, `invitations`, `role_permissions`, `role_default_permissions`, `otps`, `rate_limits`
Products and stock	`products`, `inventory_adjustments`, `suppliers`, `purchases`, `purchase_items`
Production	`recipes`, `recipe_ingredients`, `bill_of_materials`, `bom_ingredients`, `production_batches`
Sales	`transactions`, `transaction_items`, `offline_sync_queue`, `receipt_verifications`
Customers	`customers`, `customer_ledger`, `wallet_transactions`, `debt_payments`, `credit_payments`, `loyalty_settings`, `loyalty_redemptions`, `customer_merges`, `customer_duplicate_dismissals`
Money	`payments`, `money_assets`, `payment_asset_ledger`, `expenses`, `expense_categories`, `business_assets`
M-Pesa	`mpesa_settings`, `mpesa_transactions`, `mpesa_till_payments`
Online store	`customer_accounts`, `online_orders`, `online_order_items`, `customer_order_schedules`, `customer_messages`, `delivery_zones`, `website_settings`, `website_articles`, `assistant_usage`
Staff operations	`shifts`, `schedules`
Notifications	`notifications`, `notification_preferences`, `device_fcm_tokens`
Feedback	`complaints`, `complaint_replies`
Settings	`receipt_settings`, `printer_settings`, `sms_settings`, `whatsapp_settings`, `ai_settings`, `integration_configurations`
Analytics and logs	`audit_logs`, `ai_insights`, `dashboard_analytics_snapshots`, `scheduled_reports`, `report_tracked_items`, `crash_reports`
Backups and files	`google_sheets_backup`, `backup_history_logs`, `storage_files`

16.3 Enumerated types

Type	Values
`membership_status`	Active, Pending, Suspended
`invitation_status`	Pending, Accepted, Expired, Revoked
`customer_tier`	Bronze, Silver, Gold
`transaction_status`	Synced, Offline_Pending
`inventory_adjustment_type`	Restock, Damage, Reconciliation
`shift_status`	Active, Closed
`schedule_repeat`	None, Daily, Weekly, Monthly
`mpesa_status`	Pending, Completed, Failed
`notification_priority`	low, medium, high, critical
`notification_action_type`	navigate, url, none
`report_schedule_frequency`	daily, weekly, monthly
`backup_status`	success, failed, running
`sentiment`	positive, neutral, negative

16.4 What happens when a sale is saved

`fn_process_completed_transaction` runs after each sale is inserted:

1. **Loyalty**: for a registered customer, adds `floor((total − delivery fee) / 100)` points and one purchase.
2. **Shift**: adds the sale to the employee's active shift count and total.
3. **Money asset**: unless paid on credit, adds the total to the money asset for its payment method and writes a ledger line with the balance before and after.
4. **Debt**: for a credit sale, adds to the customer's debt.

`fn_process_transaction_item` reduces stock for each item, and `fn_check_low_stock` raises notifications when stock reaches its minimum.

16.5 Main database functions

Area	Functions
Business setup	`create_business_with_owner`, `check_owner_exists`, `accept_invitation_with_token`, `get_invitation_by_token`
Access	`has_permission`, `fn_user_has_permission`, `assert_permission`, `assert_business_member`, `is_owner_or_admin`, `get_user_business_ids`
Sales and payments	`process_customer_transaction`, `process_business_payment`, `void_business_payment`, `get_business_payments`, `get_money_assets`

Area	Functions
Wallet, debt, points	`process_wallet_topup`, `process_wallet_spend`, `process_debt_payment`, `settle_customer_debt_fifo`, `redeem_customer_points`, `spend_points_at_till`, `refund_till_points`
M-Pesa	`log_mpesa_payment`, `record_till_payment`, `handle_till_payment`, `confirm_store_payment_request`, `begin_order_payment`, `confirm_online_order_payment`
Stock	`adjust_stock_manually`, `adjust_stock_bulk`, `update_product_category_bulk`
Customers	`find_duplicate_customers`, `dismiss_duplicate_customers`, `merge_customers`, `calculate_customer_tier`
Customer portal	`get_public_storefront`, `place_online_order`, `cancel_my_order`, `save_order_schedule`, `delete_order_schedule`, `get_my_rewards`, `redeem_my_points`, `update_my_account`, `get_my_contact_preferences`, `update_my_contact_preferences`, `submit_website_feedback`, `run_customer_portal_daily`
Staff portal tools	`staff_update_online_order`, `staff_assign_rider`, `staff_clear_order_attention`, `staff_send_customer_message`, `get_my_deliveries`, `rider_update_delivery`
Notifications	`fn_notification_catalog`, `fn_notification_recipients`, `get_my_notification_settings`, `set_my_notification_setting`, `flag_due_schedule_reminders`
Security	`otp_issue`, `otp_check`, `otp_confirm_signup`, `check_rate_limit`, `report_crash`
Dashboard	`get_business_dashboard_metrics`, `refresh_dashboard_snapshot`
Backups	`run_due_backups`

16.6 Views

`v_active_customers`, `v_active_products`, `v_active_shifts`, `v_customer_ledger_detail`, `v_daily_sales_summary`, `v_expense_summary`, `v_low_stock_products`, `v_open_debts`, `v_pending_invitations`, `v_product_sales_rank`, `v_team_members`, `v_transaction_summary`, `v_unread_notifications`, `v_upcoming_schedules`.

16.7 Storage buckets

Bucket	Public	Holds
`product-images`	Yes	Product photos
`employee-avatars`	Yes	Staff profile pictures
`business-logos`	Yes	Logos and cover images
`receipt-exports`	No	Exported receipts
`expense-receipts`	No	Photos of expense receipts

16.8 Development scripts

`supabase/dev_clear.sql` wipes all data but keeps the schema; `supabase/dev_reset.sql` drops the schema and data. **Never run either against a database with real data.**

17. Edge functions reference

All functions live in `supabase/functions/<name>/index.ts`. Common helpers in `shared/` handle CORS, the caller's identity, `requirePermission()`, `enforceRateLimit()`, and AI provider calls.

Function	Caller	Auth	Purpose
`auth-otp`	Apps	Public, rate limited	Issue and check email verification and password reset codes; codes never leave the server
`ai-config`	Staff app	Owner	Read and save AI provider settings; returns masked keys
`chat`	Staff app	`ai.use`	Workspace assistant
`bi-analyze`	Staff app	`ai.insights` / `reports.view`	Structured business report
`complaints`	Staff app	`complaints.view`	Analyse complaints, draft and improve replies
`customer-assistant`	Website	Public, quota	Restricted shop assistant
`customer-checkout`	Website	Signed-in customer	STK push for own order (`pay`, `check`)
`mpesa-payments`	Staff app	`payments.mpesa`, `settings.integrations`	Till setup, registration, prompts, status checks
`mpesa-webhook`	Safaricom	Token in URL	STK results
`till-callback`	Safaricom	Token in URL	Direct till payments (C2B)
`ussd`	Africa's Talking	Token in URL	Dial-code menu
`send-email`	Apps	Signed in	Branded emails
`send-fcm`	Database webhook	Service role or `FCM_WEBHOOK_SECRET`	Push notifications
`sms-config`	Staff app	`communication.send`	Twilio settings and sending
`whatsapp-config`	Staff app	`communication.send`	WhatsApp settings and sending
`backup-config`	Staff app, `pg_cron`	Owner, service role	Google Sheets backup settings and runs
`sync`	Staff app	`pos.create_sale`	Offline sale queue

Deploy one with `supabase functions deploy <name>`, or all with `supabase functions deploy`.

18. Security model

18.1 Principles

- **Row-level security on every table.** No table is readable or writable without a policy.
- **Least privilege on functions.** Functions are revoked from `PUBLIC` and granted only to the roles that need them (migration `20261105002`). Service-only functions such as `run_due_backups` are granted to `service_role` alone.
- **Tenant checks inside functions.** Every `SECURITY DEFINER` function confirms the caller belongs to the business and holds the permission before acting.
- **Server-side pricing and balances.** Clients never send totals that are trusted.
- **Secrets on the server.** Provider keys are Supabase secrets or rows only functions read; the app receives masked values.

18.2 Inbound calls from providers

Safaricom and Africa's Talking can't sign in, so `mpesa-webhook`, `till-callback` and `ussd` require a secret token in the URL. Treat those URLs as passwords and change the token if one leaks. Restricting by the providers' published IP ranges is a worthwhile later step.

18.3 Sign-in codes

`auth-otp` generates six-digit codes with rejection sampling so all 900,000 codes are equally likely, stores them server-side, and limits attempts: 30 per 10 minutes per IP, 15 per hour per email, and 5 wrong guesses per code.

18.4 Rate limits

Enforced by `enforceRateLimit` in `supabase/functions/shared/cors.ts`, backed by `rate_limits` and `check_rate_limit`.

Action	Limit
AI (chat, complaints, BI)	20 per minute per person; 500 per day per business
Sign-in codes	30 per 10 minutes per IP; 15 per hour per email; 5 wrong guesses per code
Email	30 per hour per person
SMS	200 per hour per person; 1,000 per day per business
WhatsApp	200 per hour per person; 1,000 per day per business
M-Pesa prompts (staff)	60 per hour per person; 600 per day per business
Website checkout	6 per 10 minutes per customer
Dial code	40 per 10 minutes per phone

18.5 Audit

`audit_logs` records sensitive actions. Changes to sensitive employee fields are audited by `fn_employee_sensitive_audit`. Customer balance changes outside the proper functions are refused (`fn_guard_customer_balances`).

18.6 Devices

Signing keys never enter the repository: the Android key is in `~/.config/kaykays-signing/` and the desktop updater key in `~/.tauri/`. Updates are verified before installing (section 21). Shared terminals can be locked down under **Settings > Local terminal security**.

18.7 Known exposures

- Any member can read colleagues' names and contact details from `employees`, because the app needs them for assigning and display.
- Owners and Admins bypass permission checks by design.
- The `.env` file and `my-release.keystore` exist in the working folder. Confirm neither holds live production secrets before the repository is shared further, and keep `.env` out of git.

19. Local development setup

19.1 Prerequisites

- Node.js 22 (CI uses 22; 20+ works)
- Rust and Cargo, for desktop builds
- Java 21 (Temurin) and the Android SDK, for Android builds
- Supabase CLI, for backend work
- On Linux, the WebKitGTK and related packages Tauri needs (see `build.yml`)

19.2 Configure

Copy `.env.example` to `.env` and fill in at least:

```
VITE_SUPABASE_URL=https://<project-ref>.supabase.co
VITE_SUPABASE_ANON_KEY=<anon key>
VITE_AI_NAME=Kim
VITE_WEBSITE_SLUG=kaykays
```

Only `VITE_` variables reach the browser. Server secrets (M-Pesa, USSD, SMTP, service role key) are set with `supabase secrets set`, never as `VITE_` variables. CI refuses to build an installer without the Supabase URL and key, because it couldn't connect.

19.3 Run

```
npm install
npm run dev          # staff app, http://localhost:3001
npm run tauri:dev    # staff app in a desktop window
npm run website:dev  # customer website, http://localhost:5180
npm run android      # debug APK on a connected device or emulator
```

19.4 Backend changes

```
supabase login
supabase link --project-ref <project-ref>
supabase migration new <name>     # write SQL in the new file
supabase db push                  # apply to the linked project
supabase functions deploy <name>
```

Add a new migration for every change; never edit one that has already been applied. When changing role defaults, update `src/utils/permissions.ts` in the same change.

19.5 Running Supabase locally on a small machine

On a machine with limited memory, run the local Supabase stack from a scratch copy on separate ports and without Storage, and wait until `supabase start` finishes before running commands against it.

19.6 Before committing

```
npm run lint
npm run website:lint
npm test
```

CI runs the same checks. A local type check can pass where CI fails if React types are installed higher up the folder tree (for example in `~/node_modules`), so check suspicious results in a copy outside your home folder.

20. Building and releasing

20.1 Version numbers from commits

`.github/workflows/release-and-deploy.yml` runs on every push to `main`. `scripts/release-version.mjs` reads the commits since the last tag (preview with `node scripts/release-version.mjs`):

Commit type	Release
`feat!:` / `fix!:` or `BREAKING CHANGE`	Major (2.0.0 → 3.0.0)
`feat:`	Minor (2.0.0 → 2.1.0)
`fix:`, `perf:`, `refactor:`	Patch (2.0.0 → 2.0.1)
Anything else (`docs:`, `chore:`, `wip:`, …)	No release

Emoji commit prefixes are recognised too.

20.2 Release pipeline

1. Work out the version.
2. Type check and tests.
3. Build Windows (`.exe` and signed `.msi`), Linux (signed AppImage and `.deb`) and Android (signed APK).
4. Only if all of that passed: push the tag, generate `latest.json` (`scripts/update-manifest.mjs`), and publish the GitHub Release.

`build.yml` checks pull requests on all platforms (without update signing) and builds the APK on each push to `main`. It never publishes.

20.3 GitHub repository secrets

Secret	Value
`VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`	From the Supabase project
`ANDROID_KEYSTORE_BASE64`	`base64 -w0 ~/.config/kaykays-signing/kaykays-release.jks`
`ANDROID_KEYSTORE_ALIAS`, `ANDROID_KEYSTORE_PASSWORD`	From `credentials.env`
`GOOGLE_SERVICES_JSON`	Contents of `android/app/google-services.json`
`TAURI_SIGNING_PRIVATE_KEY`	Contents of `~/.tauri/kaykays-updater.key`
`TAURI_SIGNING_PRIVATE_KEY_PASSWORD`	Contents of `~/.tauri/kaykays-updater.password`
`VITE_UNSPLASH_ACCESS_KEY`, `VITE_AI_NAME`	Optional
`RELEASE_TOKEN`	Optional; only if a ruleset stops the built-in token pushing tags

20.4 Android signing

The release key lives outside the repository in `~/.config/kaykays-signing/` (`kaykays-release.jks` and `credentials.env`). **Back this folder up offline. If it is lost, installed apps can't be updated.** To build a signed APK locally:

```
set -a; . ~/.config/kaykays-signing/credentials.env; set +a
npm run build && npx cap sync android
cd android && ./gradlew assembleRelease
# output: android/app/build/outputs/apk/release/app-release.apk
```

Release builds use version code 10,000,000 + the version (1.1.0 becomes 10,001,000), which is above any local build's timestamp code, so releases always upgrade a local install.

20.5 Desktop signing

The updater key pair is in `~/.tauri/`; its public half is in `src-tauri/tauri.conf.json`. **Back it up with the Android key. If the private key is lost, installed desktop apps refuse every future update and must be reinstalled by hand.** Windows shows a SmartScreen warning until the MSI is signed with a code-signing certificate.

21. Automatic updates

Installed apps check `https://github.com/KIMGITO/ISMS/releases/latest/download/latest.json` 15 seconds after start, whenever the connection comes back, and every 6 hours. When a newer version exists they send a notification and show a green bar at the top while it downloads.

Platform	Behaviour
Windows, Linux AppImage	The download is verified against the signing key, then the app restarts after a 15-second countdown. It never restarts while a sale has items, and staff can press **Later**.
Linux `.deb`	Doesn't update itself; install the new `.deb`.
Android	`AppUpdatePlugin.java` downloads the APK, checks its SHA-256, and opens Android's installer. Android always asks to confirm, and the first time asks to allow installs from KayKay's.

If an update fails, a red bar says so, and **About this app** (under More on phones; in the sidebar and Profile on desktop) has **Check for updates**.

The repository must stay public, or the apps can't download releases. Version comparison is numeric (`utils/semver.ts`; tested as "App versions compare numerically"), so 2.10.0 is newer than 2.9.0.

v2.0.0 was the first release with automatic updates, so devices on older builds need v2.0.0 installed by hand once.

22. Deployment and operations

22.1 Deploying backend changes

Order matters:

1. Confirm the notifications webhook sends the service role key or `FCM_WEBHOOK_SECRET`; `send-fcm` rejects anything else.
2. Deploy edge functions, including `auth-otp`.
3. Apply the migrations (`supabase db push`).
4. Release the app. Older installs lose sign-up and password reset after step 3.

22.2 Scheduled jobs

Migration `20261105012_scheduled_jobs.sql` enables `pg_cron` and `pg_net` and registers:

Job	Schedule (cron, UTC)	Runs
`process-schedule-reminders`	`*/15 * * * *` (every 15 minutes)	`flag_due_schedule_reminders()`
`customer-portal-daily`	`0 2 * * *` (05:00 Nairobi)	`run_customer_portal_daily()`
`run-due-backups`	`5 * * * *` (hourly at :05)	`run_due_backups()`

22.3 Backups

Each business chooses a Google Sheets backup schedule in **Settings**, in Nairobi time. Automatic runs show in the backup history as `auto`. Backups only start once two Vault secrets exist. Run this in the Supabase SQL editor, using the legacy JWT service role key, because `backup-config` checks for a JWT:

```
SELECT vault.create_secret('https://<project-ref>.supabase.co', 'project_url');
SELECT vault.create_secret('<service-role-key>', 'service_role_key');
```

Supabase's own database backups remain the primary recovery path; the Sheets export is a readable copy for the business.

22.4 Monitoring

What	Where
Crashes	`crash_reports` table; Owners and Admins read their business's reports
Edge function errors	Supabase dashboard > Edge Functions > Logs
Scheduled jobs	`cron.job_run_details`
Payments needing attention	**Business Management > M-Pesa > Paid to the till** and **Online Store > Orders**
Backups	Backup history in **Settings**

22.5 Crash reports

`src/services/crashReporter.ts` sends crashes to `crash_reports` through `report_crash`. Emails, phone numbers, IDs and tokens are removed first (tested as "scrubPersonalData removes personal details from crash reports"), and identical crashes are counted rather than stored again.

22.6 Routine tasks

When	Task
Daily	Clear the **Paid to the till** queue; check flagged online orders
Weekly	Review crash reports and failed backups
After a release	Confirm a device updated; after all devices update, run `recalculate_loyalty_points.sql` if points changed
Every 6 months	Renew the Apple sign-in client secret, if Apple sign-in is on
If a callback URL leaks	Change `MPESA_CALLBACK_TOKEN` or `USSD_CALLBACK_TOKEN`, re-register the till, update the Africa's Talking channel

23. Testing

23.1 The test suite

`npm test` runs `src/test/runTests.ts` with `tsx`. It covers logic that is easy to get wrong and costly when wrong:

Area	Tests
Formatting	Short IDs, receipt numbers, customer numbers
Printing	ESC/POS command streams
Errors and privacy	`toUserMessage` keeps friendly text and hides technical text; crash reports are scrubbed of personal details
Access	Role permission checks with database overrides; database role defaults match the app
Production	BOM ingredient requirements and stock shortages; batch cancellation restock vs waste
AI	Pending action store and permission validation; Hugging Face default provider and token rotation
Customers	Merge preview matches the database rules
USSD	Menu registers, orders and checks balances
M-Pesa	Prompts accept Kenyan numbers in any common format
Navigation	Mobile nav keeps at most 5 slots, Settings off the bar, Sell centred for cashiers
Releases	Version from commit types (including emoji); update manifest lists signed installers and the APK; numeric version comparison
Services	Email service structure; notification repository initial state

23.2 Database testing

SQL changes for the till flow, merging and security hardening were tested on a scratch PostgreSQL. Before applying a migration to the shared project, run it on a local or scratch database first.

23.3 Manual checks before a release

1. Sign in as each role and confirm the screens match section 5.3.
2. Make a cash sale and an M-Pesa sale; check stock, points, shift totals and money assets.
3. Take a debt payment and a wallet top-up.
4. Print a receipt on the Bluetooth printer.
5. Turn the network off, make a sale, turn it on, and confirm it syncs.
6. On the website, place and pay for an order in the sandbox.
7. Confirm a notification arrives as a push on Android.

24. Known limitations and current status

24.1 Integration status

Every connection except Supabase is optional.

System	Used for	Without it	Status
Safaricom Daraja	M-Pesa prompts, till payments	Online orders can't be paid	Tested against a mock only
Africa's Talking	Dial-code ordering	No dial-code ordering	Tested with simulated requests; real gateway never called
WhatsApp Cloud API	Automatic WhatsApp messages	Staff open each chat by hand	Never run against Meta
Twilio	SMS from the staff app	Staff use WhatsApp or email	Never run
SMS provider in Supabase Auth	Phone sign-in codes	Phone sign-in stays hidden	Never run
Google	Website sign-in	Button explains it isn't available	Never run against real Google
Apple	Website sign-in	Button hidden	Never run
Resend or SendGrid	Email	Sign-in and message emails fail	Never run
Google Sheets	Backups	No Sheets backup	Scheduling tested; a real backup hasn't run
Firebase	Push notifications	No pushes	Never run end to end
AI providers	AI features	AI answers unavailable	Website assistant's model path hasn't run

24.2 Functional limits

- **Till payments can't be matched automatically**, because a till carries no account number. Staff assign them from the queue.
- **Refunds of online orders are manual**, in M-Pesa and the POS.
- **Duplicate customers are merged by staff only** until phone numbers can be verified by SMS. There's no undo for a merge.
- **USSD menu shows the first seven products in stock**, and a slow M-Pesa response can exceed the gateway's 10-second limit.
- **WhatsApp delivery isn't confirmed**: a send counts as done once Meta accepts it.
- **Email is limited to 30 per hour per person**, so it doesn't suit large lists.
- **The `.deb` Linux package doesn't update itself.**
- **Windows shows a SmartScreen warning** until the MSI is code-signed.
- **The delivery fee defaults to 0.** Set delivery areas and prices before launching the website.

24.3 Status as of 6 November 2026

- The staff app v2.0.0 is released through CI with automatic updates. No device has yet updated from one release to the next.
- The linked Supabase project is a development project; no production database exists yet.
- The security hardening, customer website, merging and M-Pesa till work are in the repository. They must be deployed in the order in section 22.1.
- Several older files (`APP_DOCUMENTATION.*`, parts of the root and `supabase/` READMEs) mention functions that no longer exist, such as `twilio-sms`, `mpesa-callback` and `gemini-copilot`. This document supersedes them.

25. Troubleshooting

Symptom	Likely cause	Fix
Installer starts but can't connect	Built without `VITE_SUPABASE_URL` / `VITE_SUPABASE_ANON_KEY`	Set the GitHub secrets and rebuild
`tauri build` fails after a dependency update	Tauri npm and crate versions differ	Pin `@tauri-apps/*` to the crates' minor version
CI type check fails but local passes	React types found higher up the folder tree locally	Check in a copy outside your home folder
Desktop app refuses an update	Signed with a different key, or key secret invalid	Restore `~/.tauri/` key; re-set `TAURI_SIGNING_PRIVATE_KEY`
Android build fails at signing	`ANDROID_KEYSTORE_BASE64` isn't valid base64	Re-create with `base64 -w0` and re-set the secret
Android won't install the update	"Install unknown apps" not allowed for KayKay's	Allow it when Android asks
No push notifications	Webhook doesn't send the service role key or `FCM_WEBHOOK_SECRET`; or no Firebase config	Fix the webhook header; set `FIREBASE_SERVICE_ACCOUNT`
M-Pesa prompt sent but sale not recorded	`mpesa-webhook` not deployed or token mismatch	Deploy it; check `MPESA_CALLBACK_TOKEN`
Till payments never appear	Till not registered, or wrong C2B short code	Press **Register till with Safaricom**; set `MPESA_C2B_SHORTCODE`
Dial-code callers see a network error	Reply took longer than 10 seconds	Check M-Pesa response time; retry
Website shows nothing	Website name not set	Set it to `kaykays` in **Online Store > Website**
Website pages 404 on refresh	Host doesn't serve `index.html` for unknown paths	Add a rewrite rule
Backups never run	Vault secrets missing	Create `project_url` and `service_role_key` (section 22.3)
"Too many requests"	Rate limit reached	Wait; see section 18.4
Sign-up or password reset stopped working	Migrations applied before `auth-otp` was deployed, or old app	Deploy `auth-otp`; update the app
Bluetooth printer missing	Not paired in Android settings	Pair it, then reopen printer settings
Customer's balance change refused	Record was merged	Refresh the app so it uses the kept record

Appendix A. Environment variables and secrets

A.1 App build variables (`.env`, GitHub secrets)

Variable	Used by	Purpose
`VITE_SUPABASE_URL`	Staff app, website	Supabase project URL
`VITE_SUPABASE_ANON_KEY`	Staff app, website	Public anon key
`VITE_AI_NAME`	Staff app	Assistant name (default Kim)
`VITE_UNSPLASH_ACCESS_KEY`	Staff app	Optional product photo search
`VITE_WEBSITE_SLUG`	Website	Which business to show (default `kaykays`)
`VITE_SITE_URL`	Website	Live address; empty means "don't index"
`VITE_AUTH_PROVIDERS`	Website	Extra sign-in options, e.g. `google,phone,apple`

A.2 Edge function secrets (`supabase secrets set`)

Secret	Purpose
`SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`	Provided by Supabase to functions
`MPESA_ENV`	`sandbox` or `production`
`MPESA_CONSUMER_KEY`, `MPESA_CONSUMER_SECRET`	Daraja app credentials
`MPESA_SHORTCODE`	Store number (production) or test paybill (sandbox)
`MPESA_PARTY_B`	Till number (production); empty in sandbox
`MPESA_PASSKEY`	Lipa na M-Pesa passkey
`MPESA_TRANSACTION_TYPE`	`CustomerBuyGoodsOnline` (till) or `CustomerPayBillOnline` (sandbox)
`MPESA_CALLBACK_TOKEN`	Authorises `mpesa-webhook` and `till-callback`
`MPESA_C2B_SHORTCODE`	Optional; short code Safaricom registers for till payments
`USSD_CALLBACK_TOKEN`	Authorises `ussd`
`USSD_SHOP_SLUG`	Business served by dial code
`FIREBASE_SERVICE_ACCOUNT`	Firebase credentials for `send-fcm`
`FCM_WEBHOOK_SECRET`	Alternative to the service role key for the webhook
`GEMINI_API_KEY` and other provider keys	Optional fallback AI keys
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`	Email delivery

A.3 Vault secrets (SQL)

Name	Value
`project_url`	`https://<project-ref>.supabase.co`
`service_role_key`	Legacy JWT service role key

A.4 Callback URLs

Provider	URL
Safaricom STK	`https://<project-ref>.supabase.co/functions/v1/mpesa-webhook?token=<MPESA_CALLBACK_TOKEN>`
Safaricom C2B	`https://<project-ref>.supabase.co/functions/v1/till-callback?type=confirmation&token=<MPESA_CALLBACK_TOKEN>`
Africa's Talking	`https://<project-ref>.supabase.co/functions/v1/ussd?token=<USSD_CALLBACK_TOKEN>`
Google / Apple sign-in	`https://<project-ref>.supabase.co/auth/v1/callback`

Appendix B. Permission codes

Defined in `src/utils/permissions.ts` and mirrored in `role_default_permissions`.

Group	Codes
Screens	`home.view`, `dashboard.view`, `complaints.view`, `complaints.reply`, `online_store.view`, `staff.schedule`
Business	`business.view`, `business.create`, `business.update`, `business.delete`, `business.switch`, `business.view_all`
Settings	`settings.view`, `settings.update`, `settings.theme`, `settings.tax`, `settings.delivery`, `settings.integrations`, `settings.storage`, `settings.security`
Staff and users	`staff.view`, `staff.invite`, `staff.update`, `staff.remove`, `staff.roles`, `users.view`, `users.manage`, `invitations.view`, `invitations.create`, `invitations.delete`
Products	`products.view`, `products.create`, `products.update`, `products.delete`, `products.import`, `products.export`, `products.manage_categories`, `products.manage_discounts`, `products.manage_tax`, `products.manage_images`
Inventory	`inventory.view`, `inventory.manage`, `inventory.receive_stock`, `inventory.adjust_stock`, `inventory.transfer_stock`, `inventory.stock_count`, `inventory.stock_history`, `inventory.wastage`, `inventory.expiry`, `inventory.import`, `inventory.export`
POS	`pos.create_sale`, `pos.manage`, `pos.open_shift`, `pos.close_shift`, `pos.discount`, `pos.override_price`, `pos.refund`, `pos.cancel_sale`, `pos.walkin_customer`
Orders	`orders.view`, `orders.create`, `orders.update`, `orders.cancel`, `orders.complete`, `orders.assign_rider`, `orders.manage`
Deliveries	`deliveries.view`, `deliveries.create`, `deliveries.assign`, `deliveries.complete`, `deliveries.cancel`, `deliveries.delivery_price`, `deliveries.delivery_zones`, `deliveries.manage`
Customers	`customers.view`, `customers.create`, `customers.update`, `customers.delete`, `customers.manage`, `customers.debts`, `customers.loyalty`, `customers.schedules`, `customers.import`, `customers.export`
Loyalty	`loyalty.view`, `loyalty.adjust`, `loyalty.configure`
Payments	`payments.view`, `payments.cash`, `payments.mpesa`, `payments.verify`, `payments.refund`, `payments.manage`
Expenses	`expenses.view`, `expenses.create`, `expenses.update`, `expenses.delete`, `expenses.export`, `expenses.manage`
Suppliers	`suppliers.view`, `suppliers.create`, `suppliers.update`, `suppliers.delete`, `suppliers.import`, `suppliers.export`, `suppliers.manage`
Purchases	`purchases.view`, `purchases.create`, `purchases.update`, `purchases.approve`, `purchases.cancel`, `purchases.print`, `purchases.import`, `purchases.export`, `purchases.manage`

Group	Codes
Production and BOM	`production.view`, `production.create`, `production.update`, `production.delete`, `production.approve`, `production.import`, `production.export`, `production.manage`, `bom.view`, `bom.create`, `bom.update`, `bom.delete`, `bom.manage`
Reports	`reports.view`, `reports.sales`, `reports.profit`, `reports.inventory`, `reports.customers`, `reports.expenses`, `reports.deliveries`, `reports.tax`, `reports.export`, `reports.manage`
Communication	`communication.view`, `communication.send`, `notifications.view`, `notifications.send`, `notifications.manage`
AI	`ai.use`, `ai.insights`, `ai.configure`, `ai.manage`
Assets and files	`assets.view`, `assets.manage`, `files.view`, `files.manage`
Audit and data	`audit.view`, `audit.manage`, `export.data`, `import.data`, `google.connect`

Appendix C. Database migrations

Applied in filename order by `supabase db push`.

Migration	Contents
`20260715001_extensions`	UUID, crypto and other extensions
`20260715002_enums`	Custom types (section 16.3)
`20260715003_core_identity_tables`	Businesses, branches, users, employees
`20260715004_access_control_tables`	Memberships, invitations, role permissions
`20260715005_product_customer_tables`	Products, customers, tiers
`20260715006_transaction_tables`	Sales, sale items, credit payments
`20260715007_operations_tables`	Shifts, adjustments, categories, schedules
`20260715008_settings_tables`	Receipt, SMS, printer and AI settings
`20260715009_extra_modules_tables`	Suppliers, purchases, recipes, assets, backups
`20260715010_analytics_log_tables`	Audit logs, snapshots, feedback, sync queue
`20260715011_indexes`	Query indexes
`20260715012_functions`	Core procedures, including the sale trigger
`20260715013_triggers`	Table triggers
`20260715014_rls_policies`	Row-level security policies
`20260715015_views_realtime`	Views and Realtime publication
`20260715016_storage_setup`	Storage buckets and policies
`20260719001_bom_production`	Bill of materials and production batches
`20260722001_password_reset_functions`	Password reset RPCs
`20260808001_customer_wallet_rpcs`	Atomic sale, wallet and debt RPCs
`20260819001_money_assets_payments`	Payments, money assets and asset ledger
`20260819002_schedule_reminder_cron`	Schedule reminder job
`20260918003_update_customer_tier_thresholds`	Tier thresholds and backfill
`20261105002_security_hardening`	Function privileges and tenant checks; server OTPs
`20261105003_rate_limits_crash_reports`	Rate limits and crash reports
`20261105010_customer_portal`	Customer website schema and RPCs
`20261105011_fix_process_business_payment`	One working payment RPC
`20261105012_scheduled_jobs`	`pg_cron` jobs, including due backups
`20261105013_customer_groups_loyalty_messaging`	Customer groups, loyalty settings, messaging consent

Migration	Contents
`20261105014_role_enforcement`	`has_permission()` and database role defaults
`20261105015_rider_deliveries_points_on_orders`	Rider deliveries, points on online orders
`20261105016_customer_merge_notification_settings`	Customer merging, notification preferences
`20261106001_void_payments_expenses_mpesa_till`	Voiding payments and expenses, M-Pesa till queue, account numbers

Appendix D. Glossary

Term	Meaning
BOM	Bill of materials: the ingredients and quantities that make one unit of a product
C2B	Customer to business: an M-Pesa payment the customer starts from their own phone
Daraja	Safaricom's M-Pesa API
Edge function	Server code running on Supabase, written for Deno
FCM	Firebase Cloud Messaging, used for push notifications
Money asset	The running balance the business holds per payment method
Paybill	An M-Pesa business number where customers enter an account number
`pg_cron`	PostgreSQL extension that runs scheduled jobs
RLS	Row-level security: PostgreSQL rules deciding which rows each user can reach
RPC	A database function called from the app
Service role	The Supabase key that bypasses RLS; only ever used on the server
`SECURITY DEFINER`	A database function that runs with its owner's rights, after its own checks
STK push	An M-Pesa prompt that appears on the customer's phone asking for their PIN
Till	An M-Pesa Buy Goods number; payments carry no account number
Tenant	One business and its data
USSD	Dial codes such as `*384*123#` that work on any phone without internet
Walk-in	A sale with no registered customer